

BML: Probabilistic Inference and HMMs

Derek Aguiar

University of Connecticut – CSE5825

Announcements

-

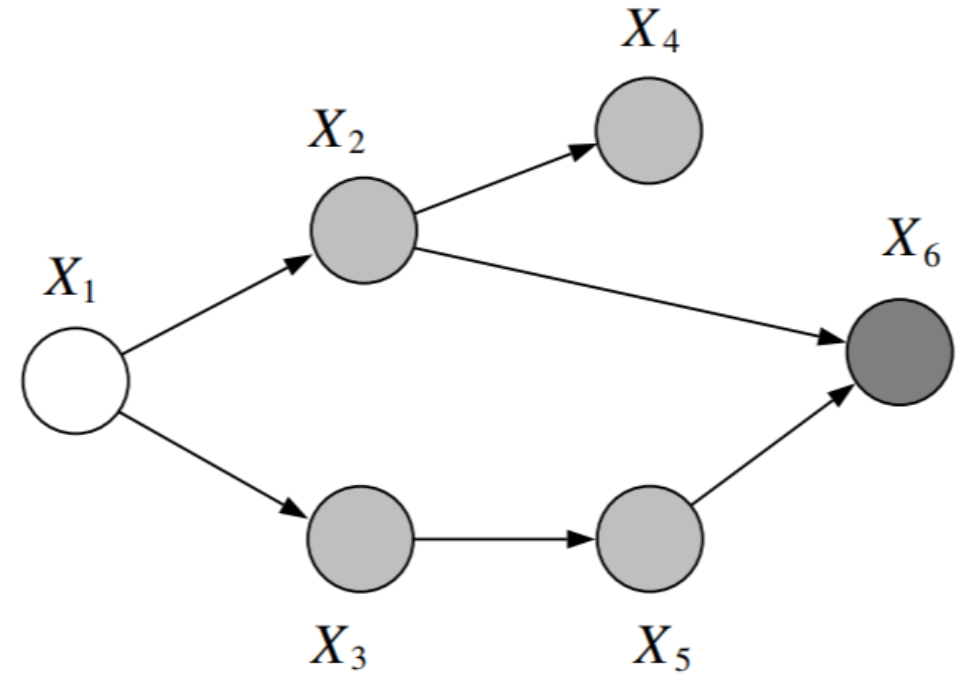
Notation

Query nodes: $X_F = \{X_1\}$

Evidence nodes: $X_E = \{X_6\}$

Nodes that are marginalized out

$X_R = V - (E \cup F) = \{X_2, X_3, X_4, X_5\}$

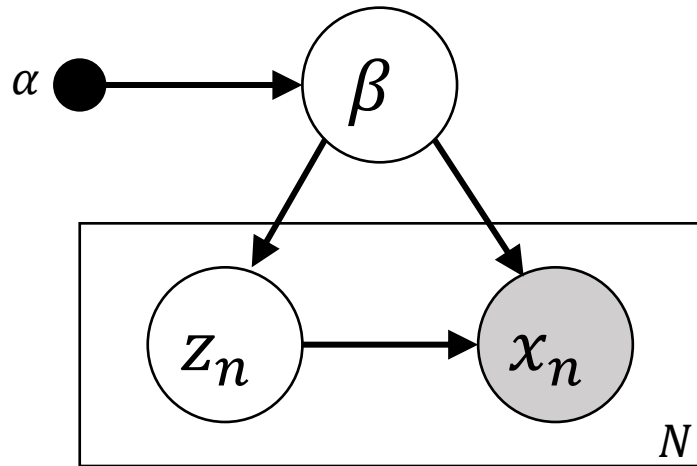


Probabilistic inference

- Let $E, F \in \{1, \dots, n\}$ be subset of node indices for graphical model G such that $E \cap F = \emptyset$
- Generally, the goal of probabilistic inference is to calculate

$$p(X_F | X_E)$$

Probabilistic inference



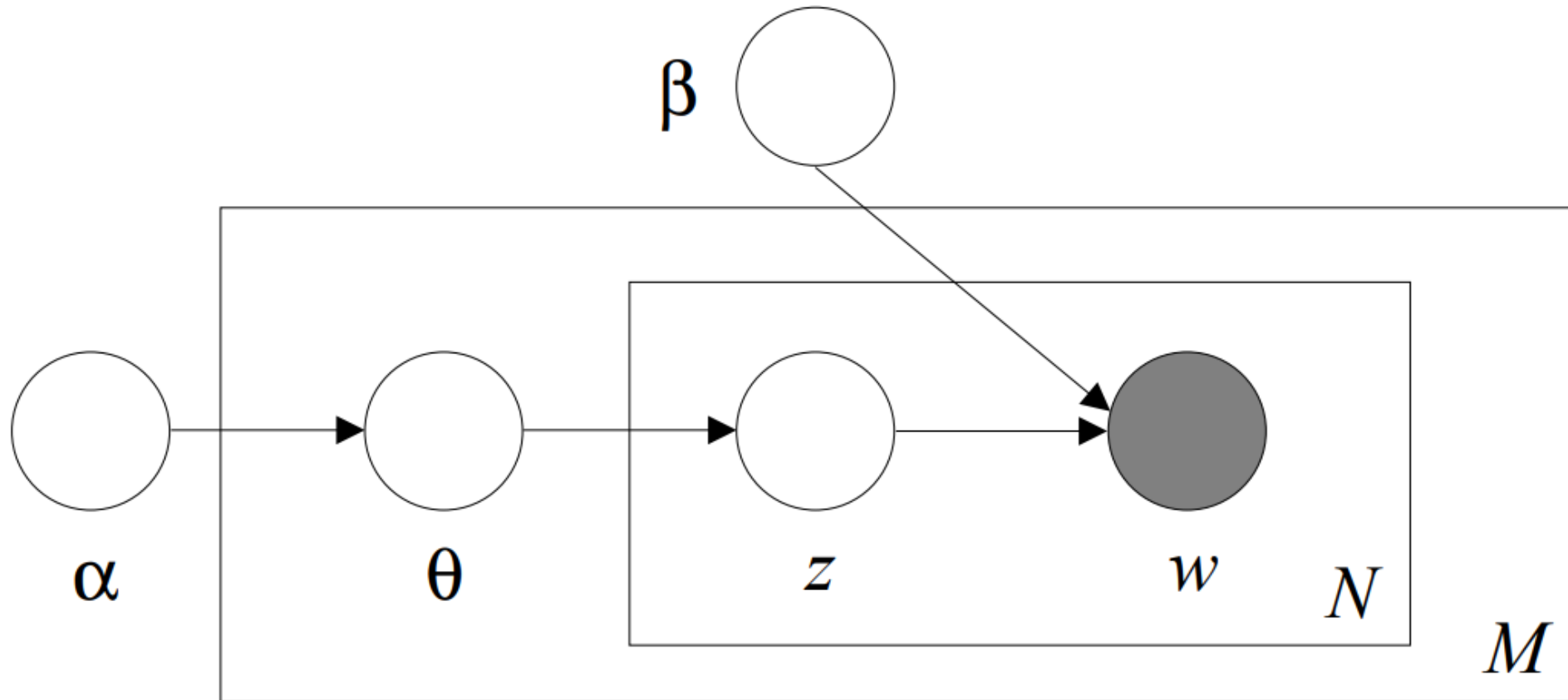
$$\begin{array}{l} \text{posterior} \\ p(z, \beta | x) \end{array} = \frac{\begin{array}{l} \text{data likelihood} \\ p(x | z, \beta) \end{array} \begin{array}{l} \text{prior} \\ p(z, \beta) \end{array}}{\int \int p(x, z, \beta) dz d\beta}$$

**marginal likelihood
or model evidence**

E.g. Naïve Bayes

What is the inference problem?

E.g. mixed membership models (LDA)



What is the inference problem?

Probabilistic inference

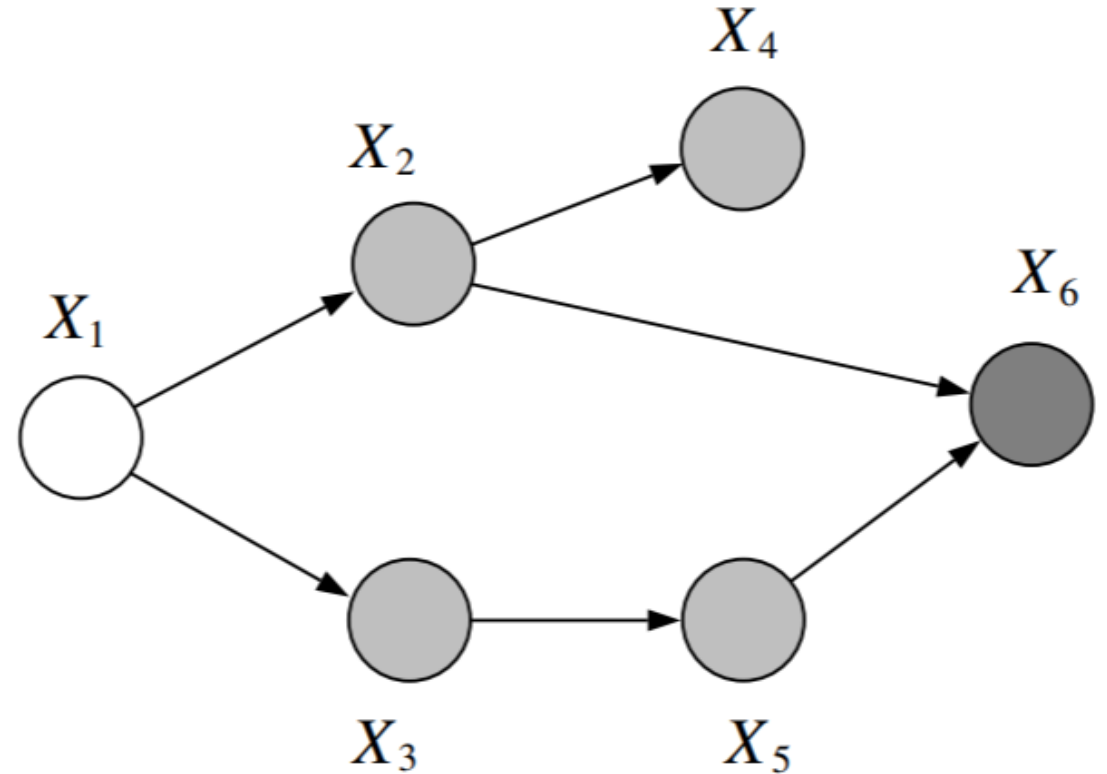
- Recall that

$$p(x_F | x_E) = \frac{p(x_E, x_F)}{p(x_E)}$$

- where

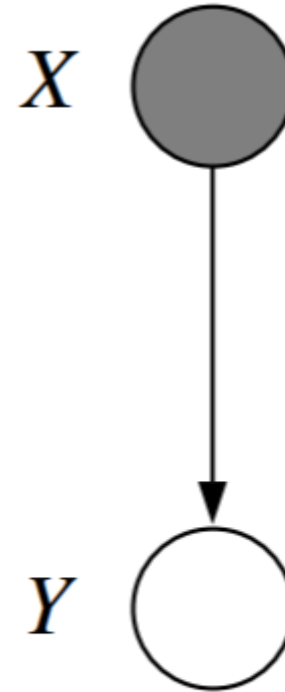
$$p(x_E, x_F) = \sum_{x_R} p(x_E, x_F, x_R)$$

$$p(x_E) = \sum_{x_F} p(x_E, x_F)$$

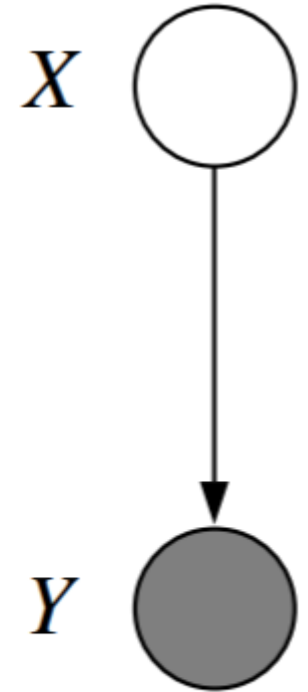


Simple cases

- Quantities of interest $p(x)$ and $p(y|x)$
- (b) Conditioning on X , computing $p(x)$ and $p(y|x)$ is easy
- (c) Conditioning on Y , requires Bayes rule



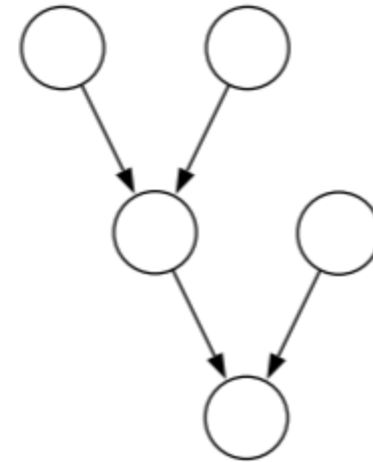
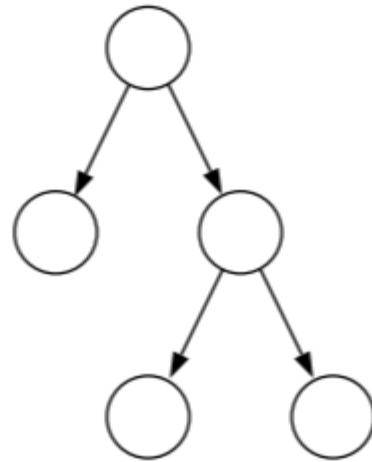
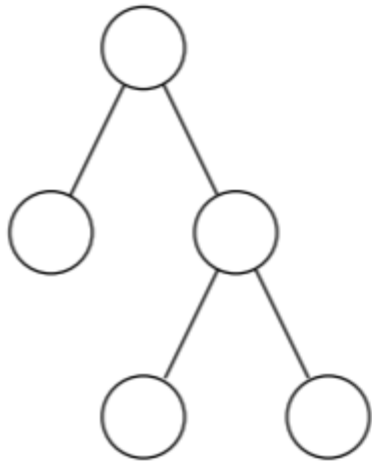
(b)



(c)

Tree graphical models

- An undirected graphical model is a tree if there exists a single path between any pair of nodes
- A directed graphical model is a tree if its moralization is a tree (no V structures)



Tree Joint Probability

- Consider undirected trees

$$p(x) = \frac{1}{Z} \left(\prod_{i \in \mathcal{V}} \psi(x_i) \prod_{(i,j) \in \mathcal{E}} \psi(x_i, x_j) \right)$$

- For directed trees we have the joint

$$p(x) = p(x_r) \prod_{(i,j) \in \mathcal{E}} p(x_j \mid x_i).$$

Parameterization for directed trees

- We can set potentials equal to these conditional probabilities

$$\psi(x_r) \triangleq p(x_r)$$

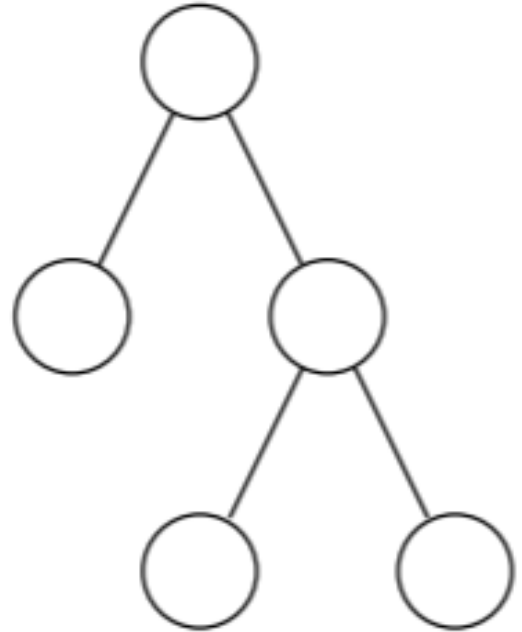
$$\psi(x_i) \triangleq 1 \quad \text{for } i \neq r$$

$$\psi(x_i, x_j) \triangleq p(x_j \mid x_i).$$

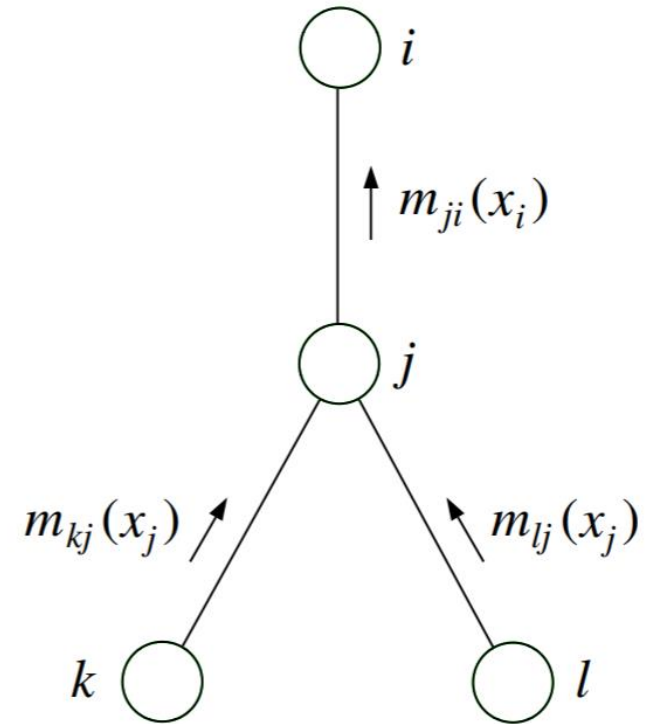
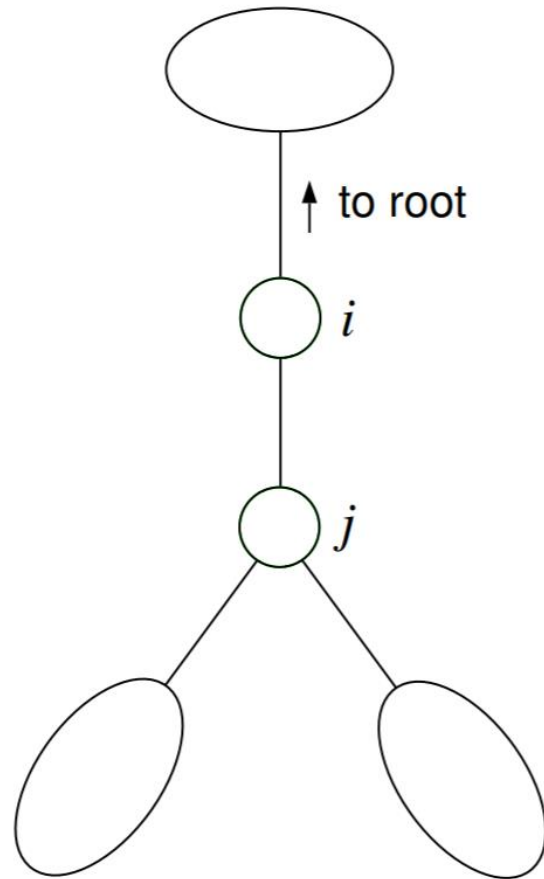
- **Directed tree special case of undirected tree**

Ordering for processing nodes

- Consider an arbitrary root. We wish to compute the probability of that node.
- Which order do we want to process nodes?
- What is the largest clique in a tree?

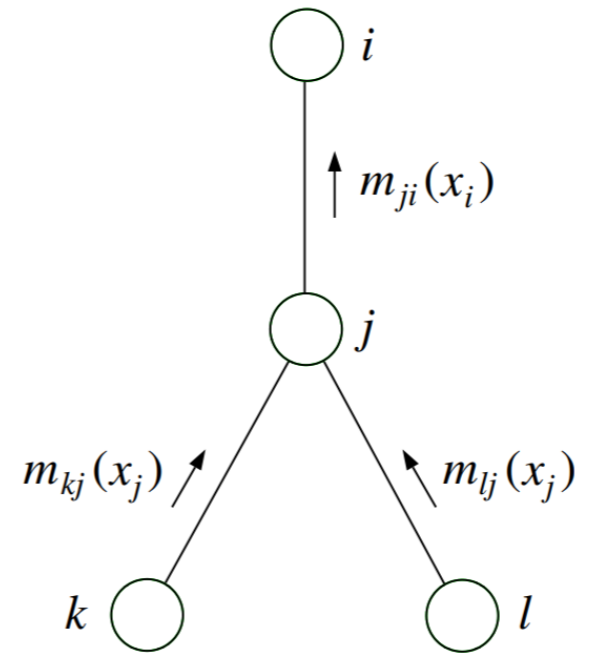


A single step (node j)



What is the message $m_{j \rightarrow i}(x_i)$?

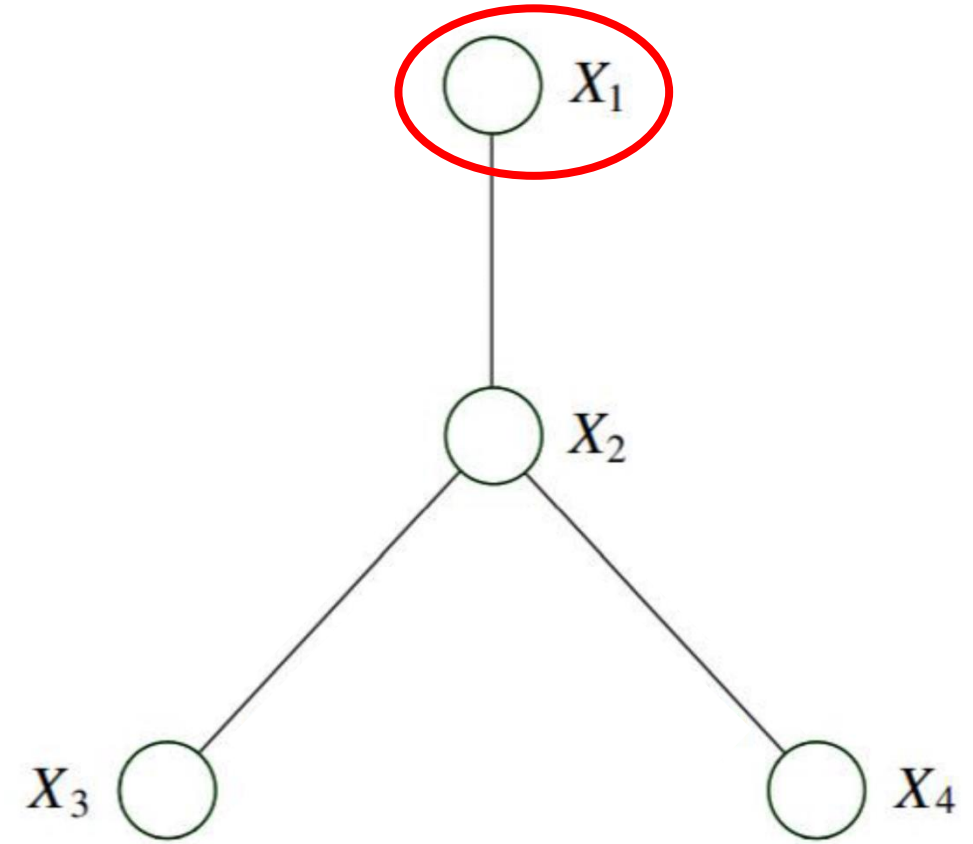
- The node potential $\psi(x_j)$
- The pairwise potential $\psi(x_i, x_j)$
- $m_{k \rightarrow j}(x_j)$ for neighbors $k \in N(j) \setminus i$



$$m_{j \rightarrow i}(x_i) \triangleq \sum_{x_j} \psi(x_j) \psi(x_i, x_j) \prod_{k \in N(j) \setminus i} m_{k \rightarrow j}(x_j)$$

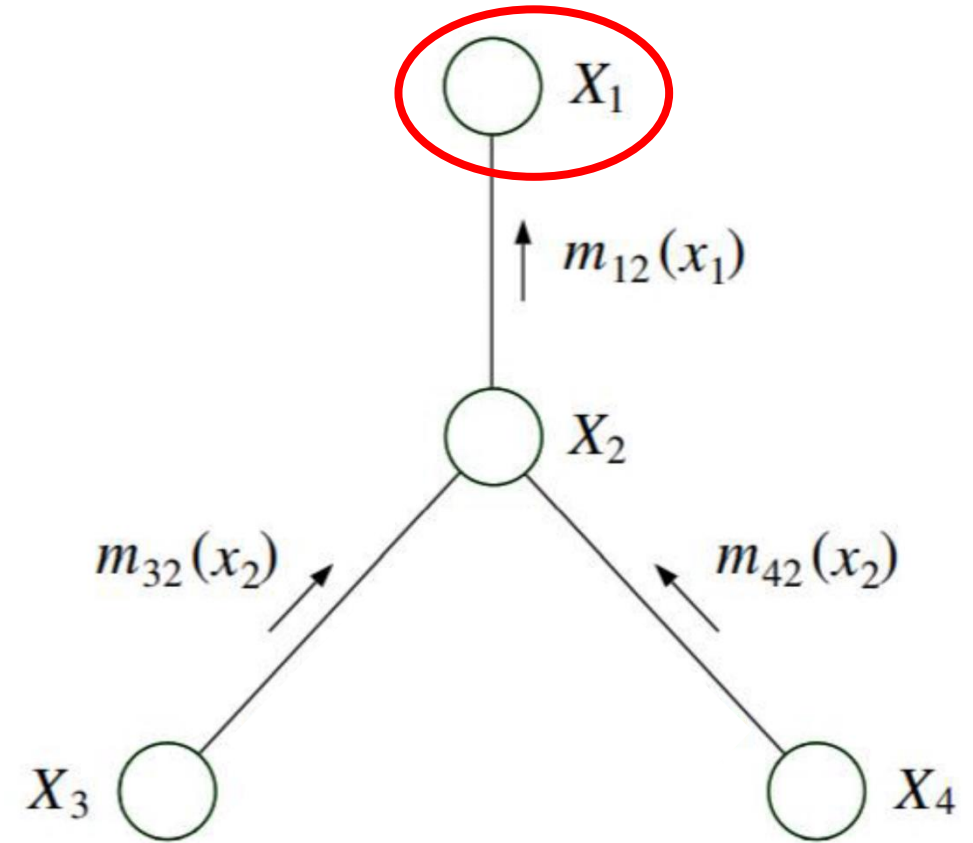
Sum product algorithm

- Pick an arbitrary node



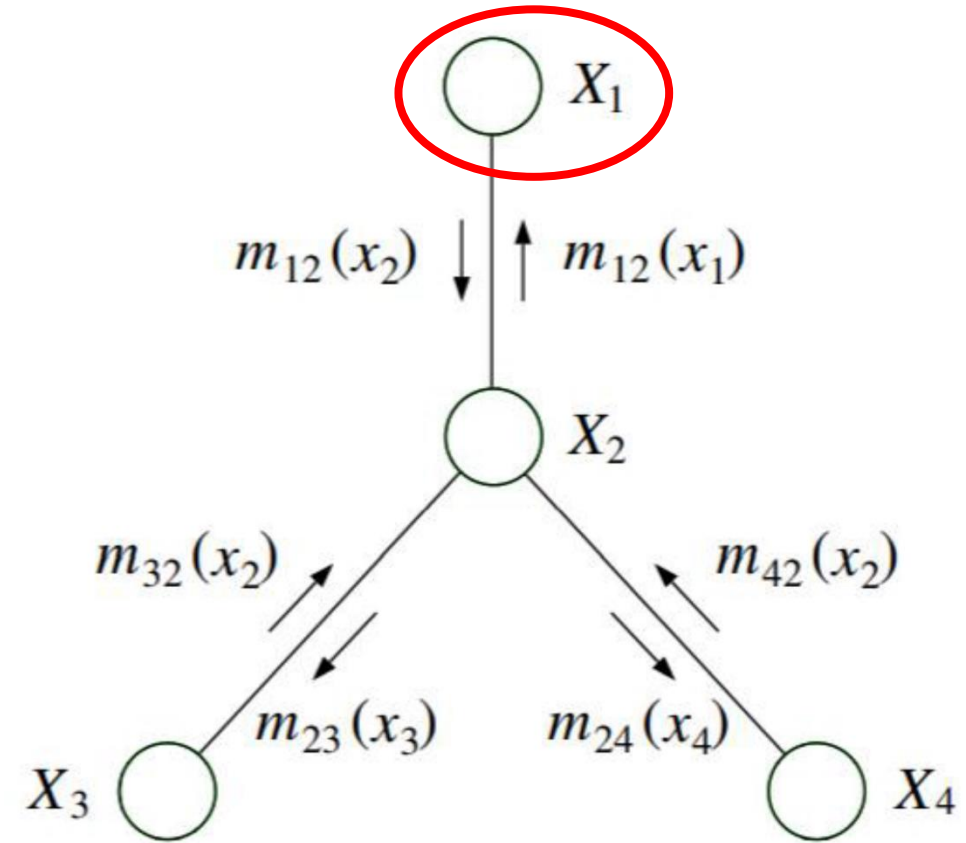
Sum product algorithm

- Pick an arbitrary node
- Compute and store messages from leaves to root



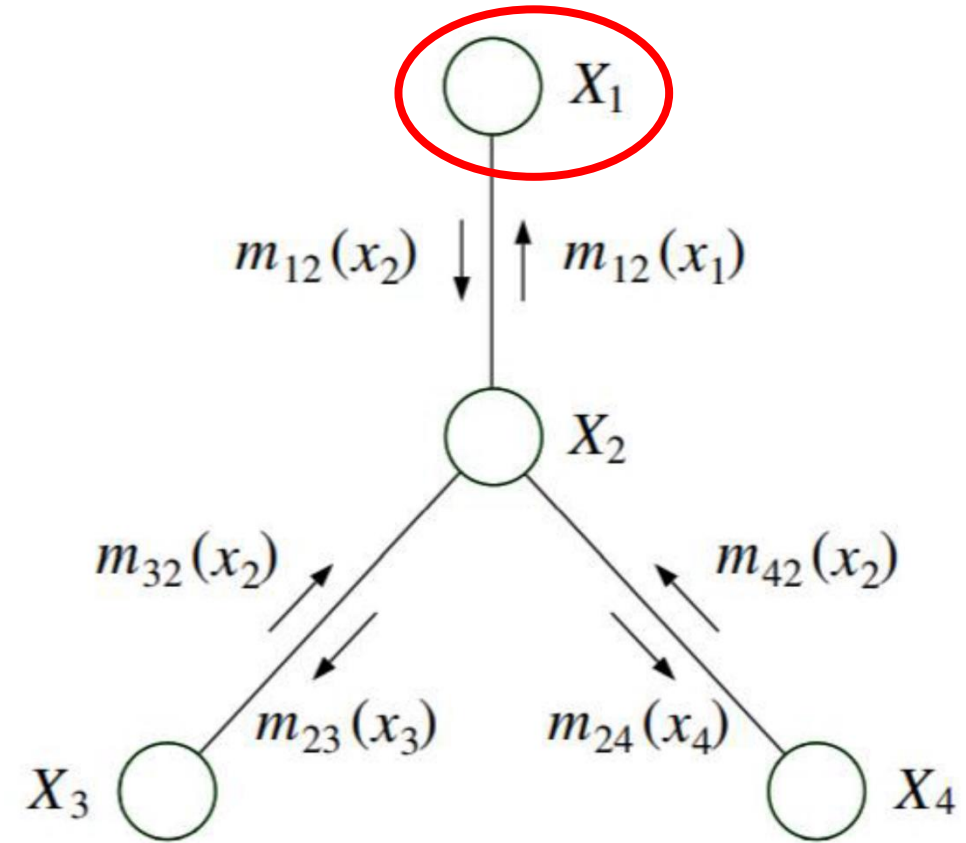
Sum product algorithm

- Pick an arbitrary node
- Compute and store messages from leaves to root
- Compute and store messages from root to leaves



Sum product algorithm

- Pick an arbitrary node
- Compute and store messages from leaves to root
- Compute and store messages from root to leaves
- Compute marginals
 - $X_r = X_1$
 - Combined steps
 $m_{3 \rightarrow 2}, m_{4 \rightarrow 2}, m_{2 \rightarrow 1}, p(x_1),$
 $m_{1 \rightarrow 2}, p(x_2), m_{2 \rightarrow 3}, p(x_3),$
 $m_{2 \rightarrow 4}, p(x_4)$



Example: Markov Model

- Markov model is a chain of random variables
- Parameters $p(z_{t+1}|z_t)$ give the probability of the next observation given the current observation

n – *gram* model for documents

- Consider a model for generating phrases
- Applications
 - Aiding translation
 - $p(\text{"green eggs and ham"}) > p(\text{"green eggs and spam"})$
 - Correcting spelling
 - $p(\text{"normal distribution"}) > p(\text{"normal distrbuton"})$
 - Grammar and speech correction
 - $p(\text{"directions to UConn"}) > p(\text{"directions to you can"})$

How do we model $p(z_0 = \text{directions}, z_1 = \text{to}, z_2 = \text{UConn})$?

n – *gram* model for documents

- What are the random variables?

n – *gram* model for documents

- What are the random variables?
- What does the model specify?

n – *gram* model for documents

- Random variables are vocabulary terms $(z_1, \dots, z_V)$
- The model specifies the probability of the next term given the previous term.
- [unigram, bigram, n-gram models here]

The difference n-makes

1

gram

–To him swallowed confess hear both. Which. Of save on trail for are ay device and rote life have

–Hill he late speaks; or! a more to leg less first you enter

2

gram

–Why dost stand forth thy canopy, forsooth; he is this palpable hit the King Henry. Live king. Follow.

–What means, sir. I confess she? then all sorts, he is trim, captain.

3

gram

–Fly, and will rid me these news of price. Therefore the sadness of parting, as they say, 'tis done.

–This shall forbid it should be branded, if renown made it empty.

4

gram

–King Henry. What! I will go seek the traitor Gloucester. Exeunt some of the watch. A great banquet serv'd in;

–It cannot be but so.

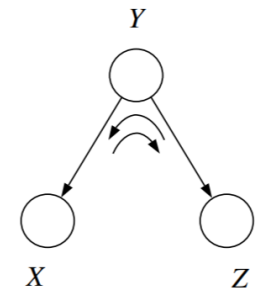
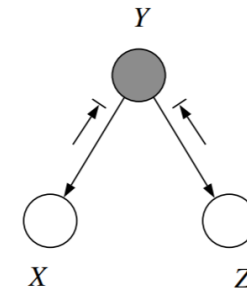
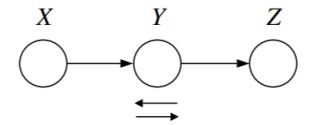
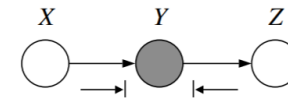
(aside) Claude Shannon's bigram model

- In Claude Shannon's seminal 1948 paper inventing information theory, he also invented the bigram model
- Procedure:
 - Choose a random book from a shelf, open to a random page, choose a random word, and record it as $\bar{z}_0$
 - Choose another book, open a random page, read it until $\bar{z}_0$ is found, record the next word as $\bar{z}_1$
 - Repeat this process for T steps

What is this procedure doing?

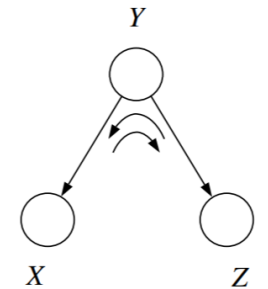
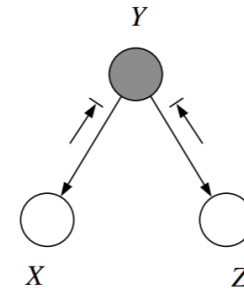
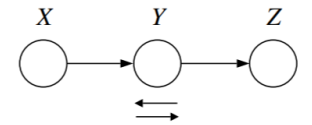
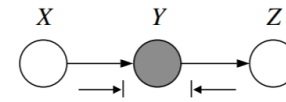
Hidden Markov Models

- HMMs make conditional independence assumptions



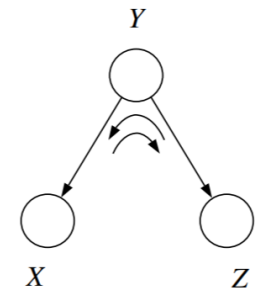
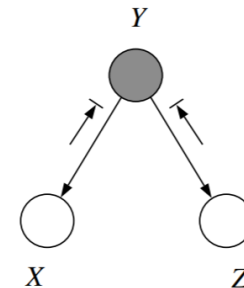
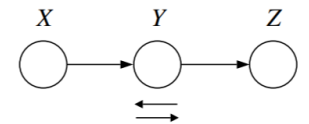
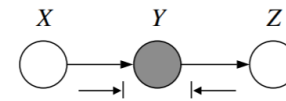
Hidden Markov Models

- HMMs make conditional independence assumptions
 - Observations are conditionally independent given the states
 - Observations are marginally dependent
- To specify the joint, we must specify



Hidden Markov Models

- HMMs make conditional independence assumptions
 - Observations are conditionally independent given the states
 - Observations are marginally dependent
- To specify the joint, we must specify
 - Transition probabilities $p(z_{t+1}|z_t)$
 - Observation probabilities $p(x_t|z_t)$
 - Initial state probability $p(z_1)$



Hidden Markov Models

- Compute: marginal probabilities or Maximum a posteriori estimates
 - $P(\text{loci 100 is in gene sequence})$
 - Input: audio signal, Output: sequence of words with max probability
- The HMM is a directed tree, we turn it into an undirected tree with potentials:

$$\psi(z_{t-1}, z_t) \triangleq p(z_t | z_{t-1})$$

$$\psi(z_t, x_t) \triangleq p(x_t | z_t)$$

$$\psi(z_1) \triangleq p(z_1)$$

$$\psi(z_t) \triangleq 1, \quad t > 1$$

$$\psi(x_t) \triangleq \delta_{\bar{x}_t}(x_t)$$

Messages (observed)

- Time 1

$$\begin{aligned} m_{x_1 \rightarrow z_1}(z_1) &= \sum_{x_1} \psi(x_1) \psi(x_1, z_1) \\ &= \sum_{x_1} \delta_{\bar{x}_1}(x_1) p(x_1 | z_1). \end{aligned}$$

- Time t

$$\begin{aligned} m_{x_t \rightarrow z_t}(z_t) &= \sum_{x_t} \psi(x_t) \psi(x_t, z_t) \\ &= \sum_{x_t} \delta_{\bar{x}_t}(x_t) p(x_t | z_t). \end{aligned}$$

- Each message selects the appropriate observation probability

$$m_{x_t \rightarrow z_t}(z_t) = p(\bar{x}_t | z_t)$$

Messages (forward)

- Time 1

$$\begin{aligned}m_{z_1 \rightarrow z_2}(z_2) &= \sum_{z_1} \psi(z_1) \psi(z_1, z_2) m_{x_1 \rightarrow z_1}(z_1) \\ &= \sum_{z_1} p(z_1) p(z_2 | z_1) p(\bar{x}_1 | z_1).\end{aligned}$$

- Time t

$$\begin{aligned}m_{z_t \rightarrow z_{t+1}}(z_t) &= \sum_{z_t} \psi(z_t) \psi(z_t, z_{t+1}) m_{x_t \rightarrow z_t}(z_t) m_{z_{t-1} \rightarrow z_t}(z_t) \\ &= \sum_{z_t} p(z_{t+1} | z_t) p(\bar{x}_t | z_t) m_{z_{t-1} \rightarrow z_t}(z_t).\end{aligned}$$

Messages (forward) interpretation

- The message between hidden states is simply

$$\begin{aligned}m_{z_1 \rightarrow z_2}(z_2) &= \sum_{z_1} p(z_1) p(z_2 | z_1) p(\bar{x}_1 | z_1) \\&= \sum_{z_1} p(z_1, z_2, \bar{x}_1) \\&= p(\bar{x}_1, z_2)\end{aligned}$$

- And at time t : $m_{z_{t-1} \rightarrow z_t}(z_t) = p(\bar{x}_1, \dots, \bar{x}_{t-1}, z_t).$

- Which can be seen through induction

$$\begin{aligned}m_{z_t \rightarrow z_{t+1}}(z_{t+1}) &= \sum_{z_t} p(z_{t+1} | z_t) p(\bar{x}_t | z_t) p(\bar{x}_1, \dots, \bar{x}_{t-1}, z_t) \\&= p(\bar{x}_1, \dots, \bar{x}_t, z_{t+1}).\end{aligned}$$

Messages (backward)

$$m_{z_{t+1} \rightarrow z_t}(z_t) = \sum_{z_{t+1}} \psi(z_{t+1}) \psi(z_t, z_{t+1}) m_{x_{t+1} \rightarrow z_{t+1}}(z_{t+1}) m_{z_{t+2} \rightarrow z_{t+1}}(z_{t+1})$$

Sum product also
needs messages
from time T to
time 1

$$= \sum_{z_{t+1}} p(z_{t+1} | z_t) p(\bar{x}_{t+1} | z_{t+1}) m_{z_{t+2} \rightarrow z_{t+1}}(z_{t+1}).$$

Forward-backward algorithm

- Marginal probability at a particular node

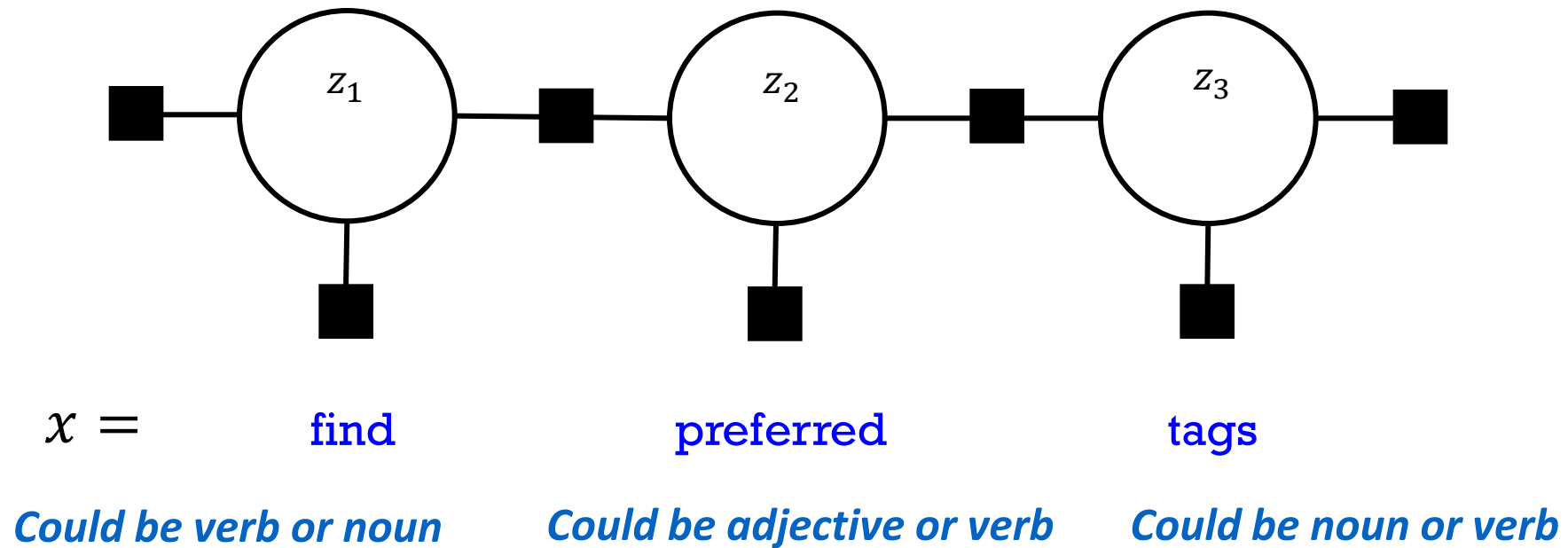
$$p(z_t) \propto m_{z_{t-1} \rightarrow z_t}(z_t) m_{z_{t+1} \rightarrow z_t}(z_t) m_{x_t \rightarrow z_t}(z_t).$$

$$= p(\bar{x}_1, \dots, \bar{x}_{t-1}, z_t).$$

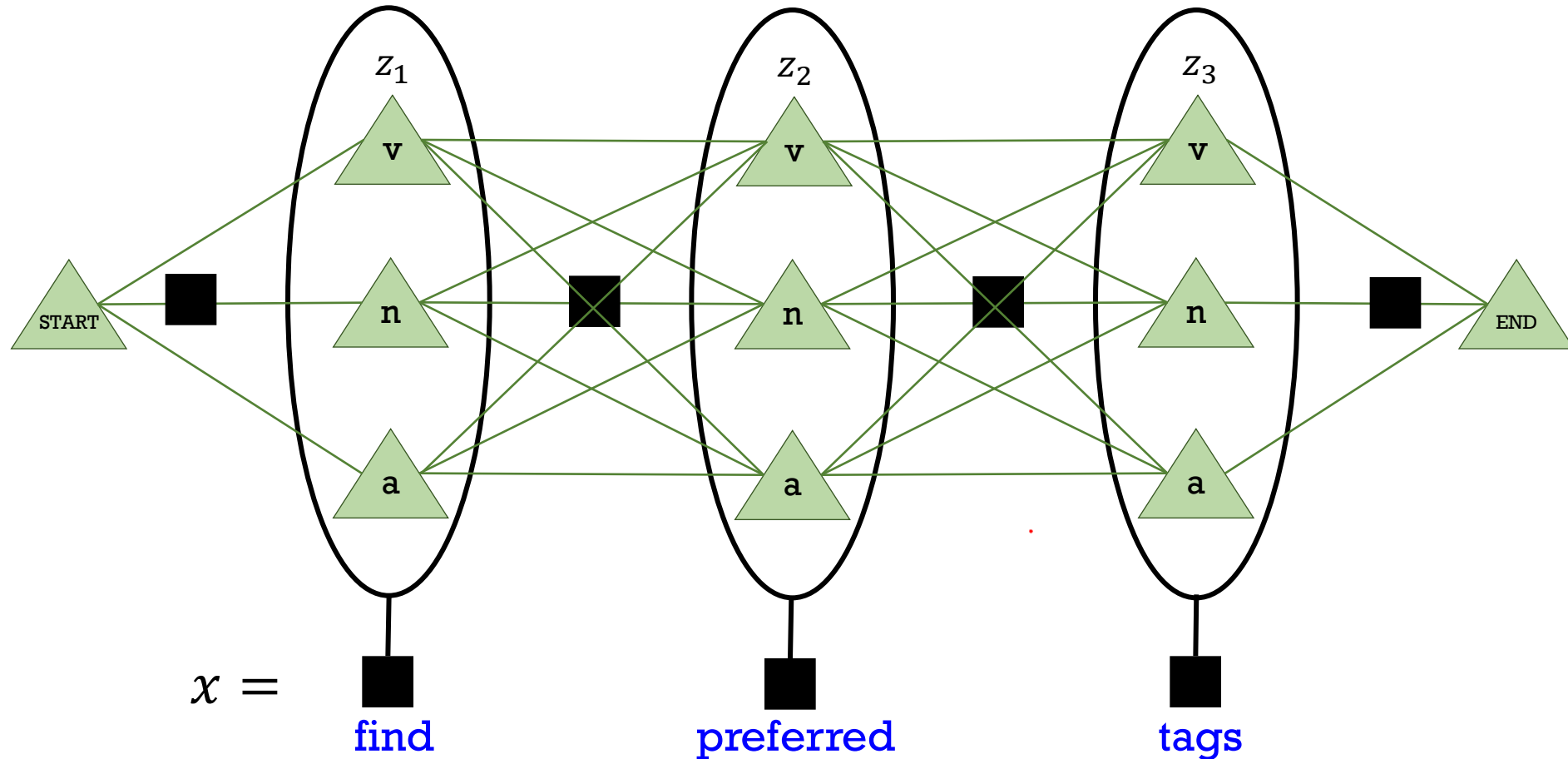
$$= p(\bar{x}_T, \dots, \bar{x}_{t+1} | z_t).$$

$$= p(\bar{x}_t | z_t)$$

So Let's Review Forward-Backward ...

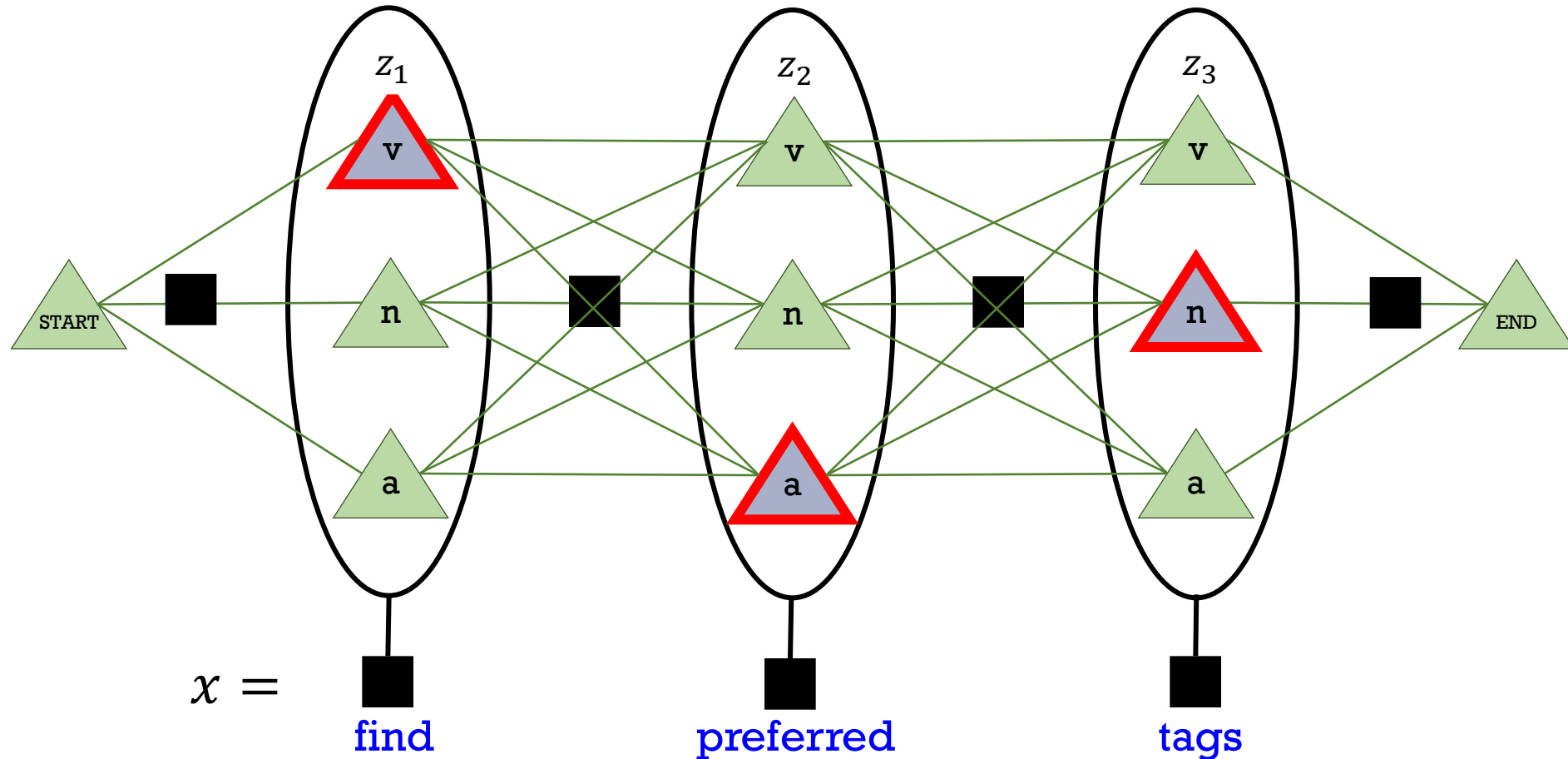


So Let's Review Forward-Backward ...



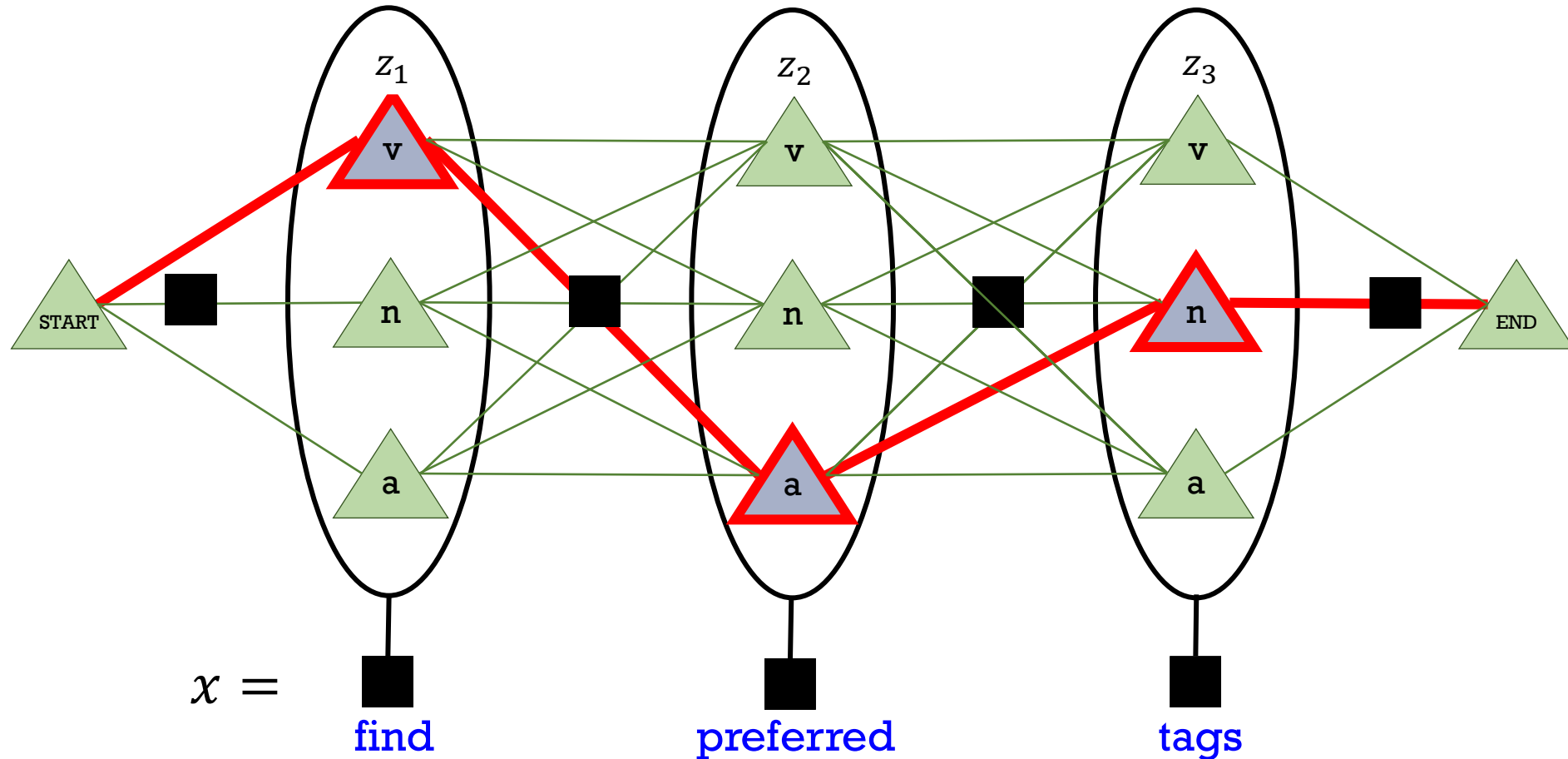
- Show the possible *values* for each variable

So Let's Review Forward-Backward ...



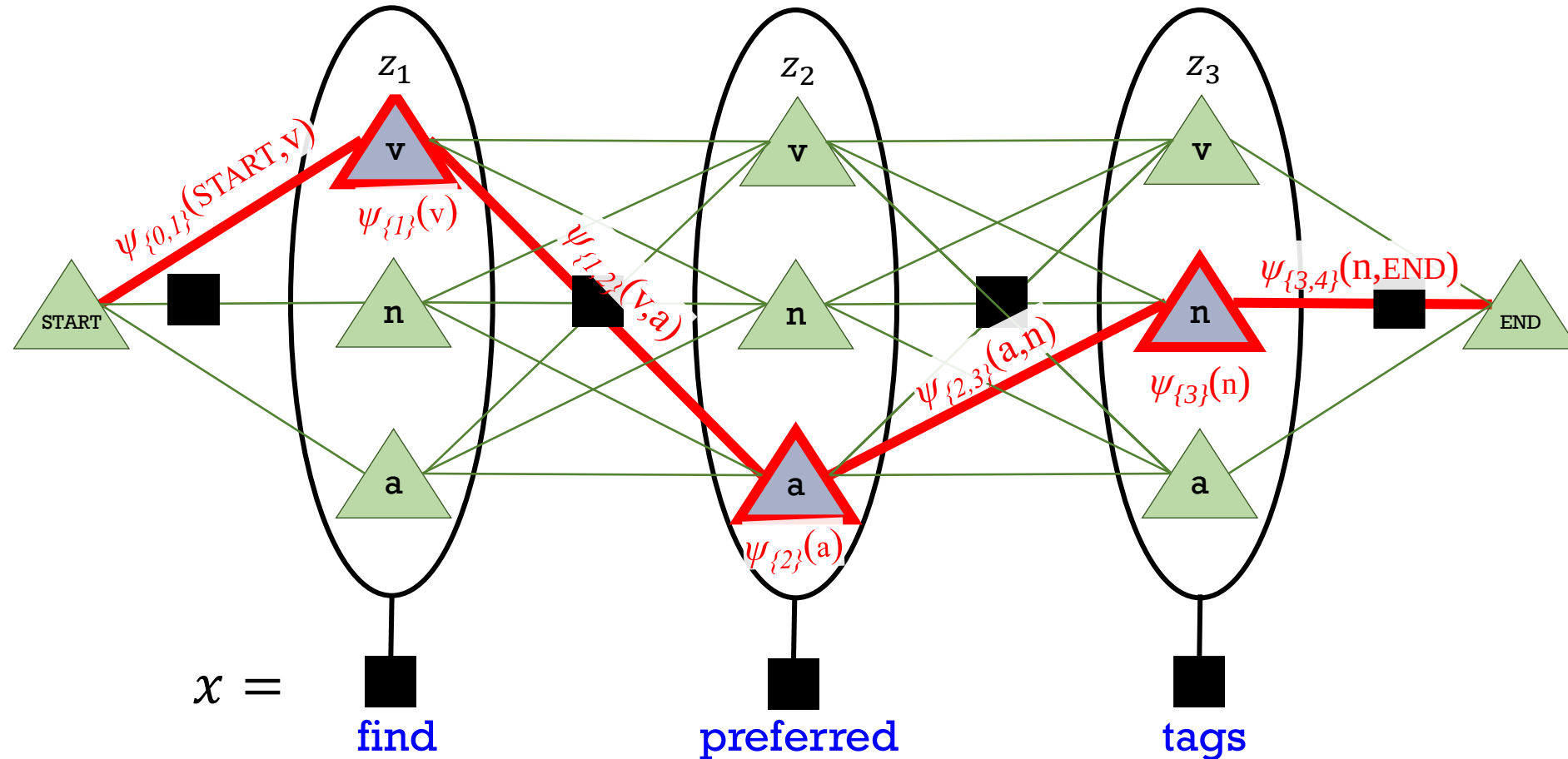
- Let's show the possible *values* for each variable
- One possible assignment

So Let's Review Forward-Backward ...



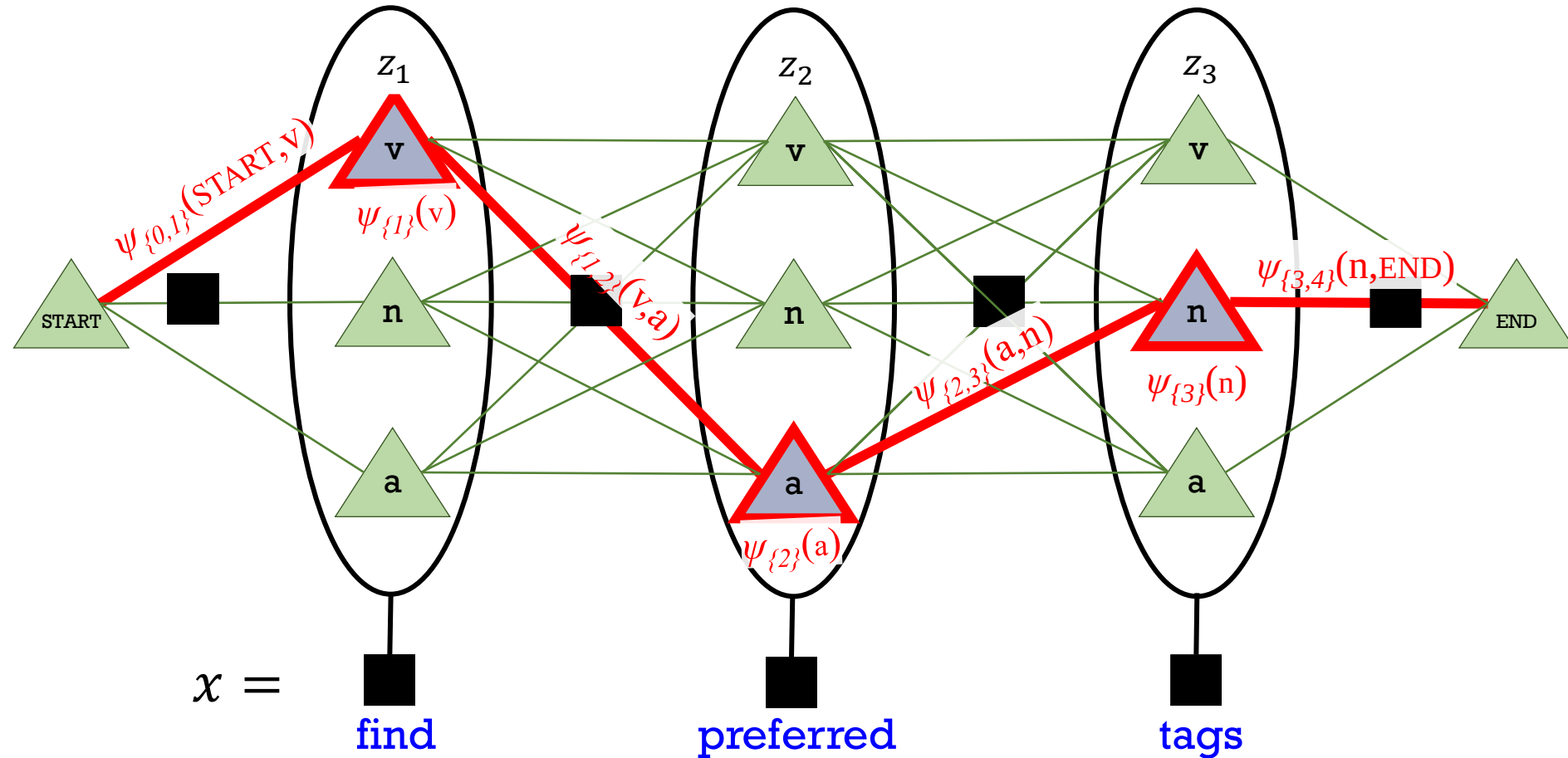
- Let's show the possible *values* for each variable
- One possible assignment
- And what the 7 factors **think of it** ...

Max-product (Viterbi) Algorithm: Most Probable Assignment



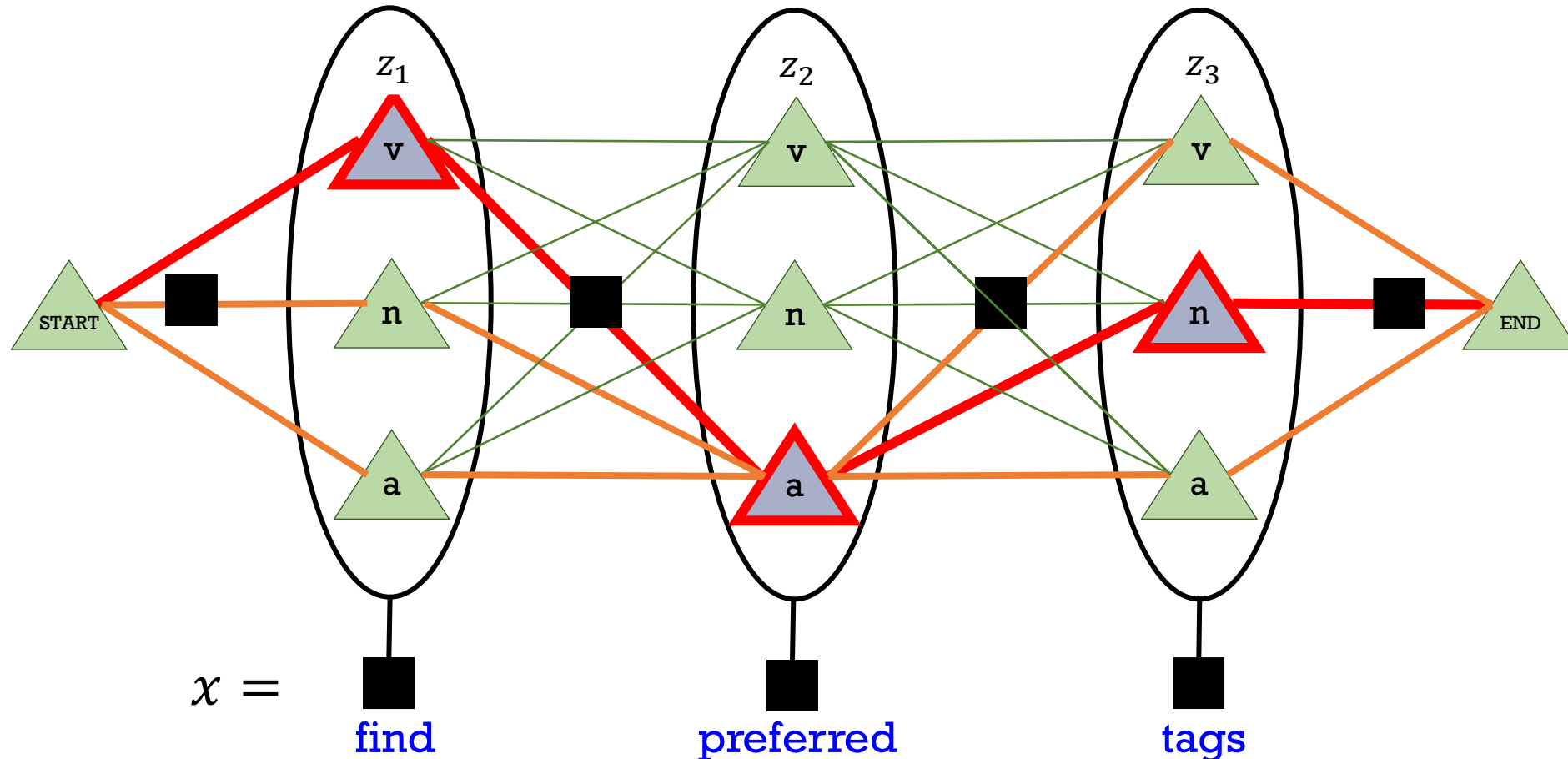
- So $p(v \ a \ n) = (1/Z) * \text{product of 7 numbers}$
- Numbers associated with edges and nodes of path
- Most probable assignment = **path with highest product**

Max-product (Viterbi) Algorithm: Most Probable Assignment

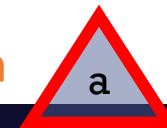


- So $p(v \ a \ n) = (1/Z) * \text{product weight of one path}$

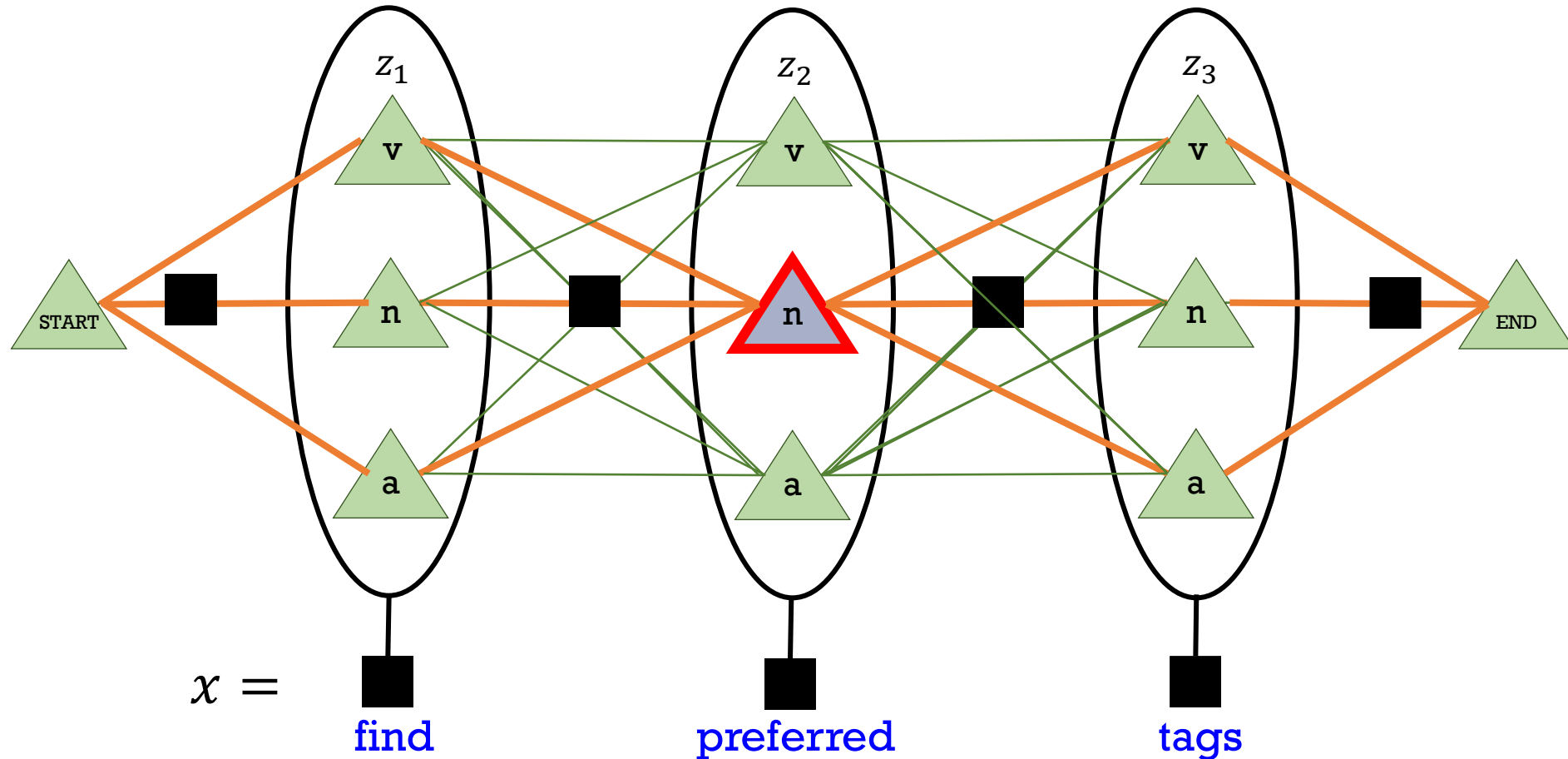
Sum Product (Forward-Backward) Algorithm: Finds Marginals



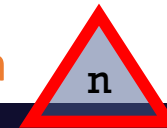
- So $p(\mathbf{v} \mathbf{a} \mathbf{n}) = (1/Z) * \text{product weight of one path}$
- Marginal probability $p(X_2 = a)$
 $= (1/Z) * \text{total weight of all paths through}$



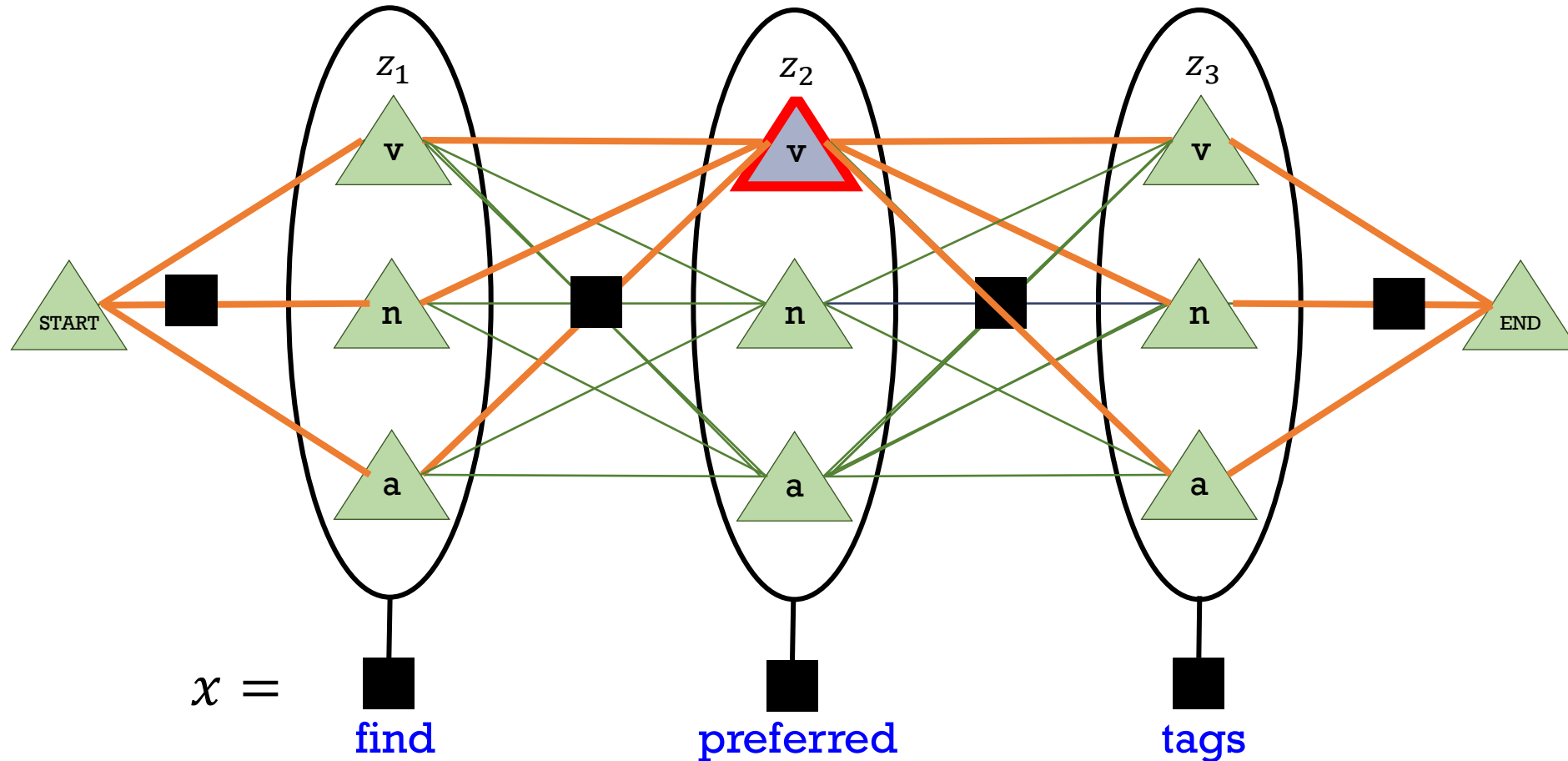
Sum Product (Forward-Backward) Algorithm: Finds Marginals



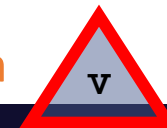
- So $p(v \ a \ n) = (1/Z) * \text{product weight of one path}$
- Marginal probability $p(X_2 = n)$
 $= (1/Z) * \text{total weight of all paths through}$



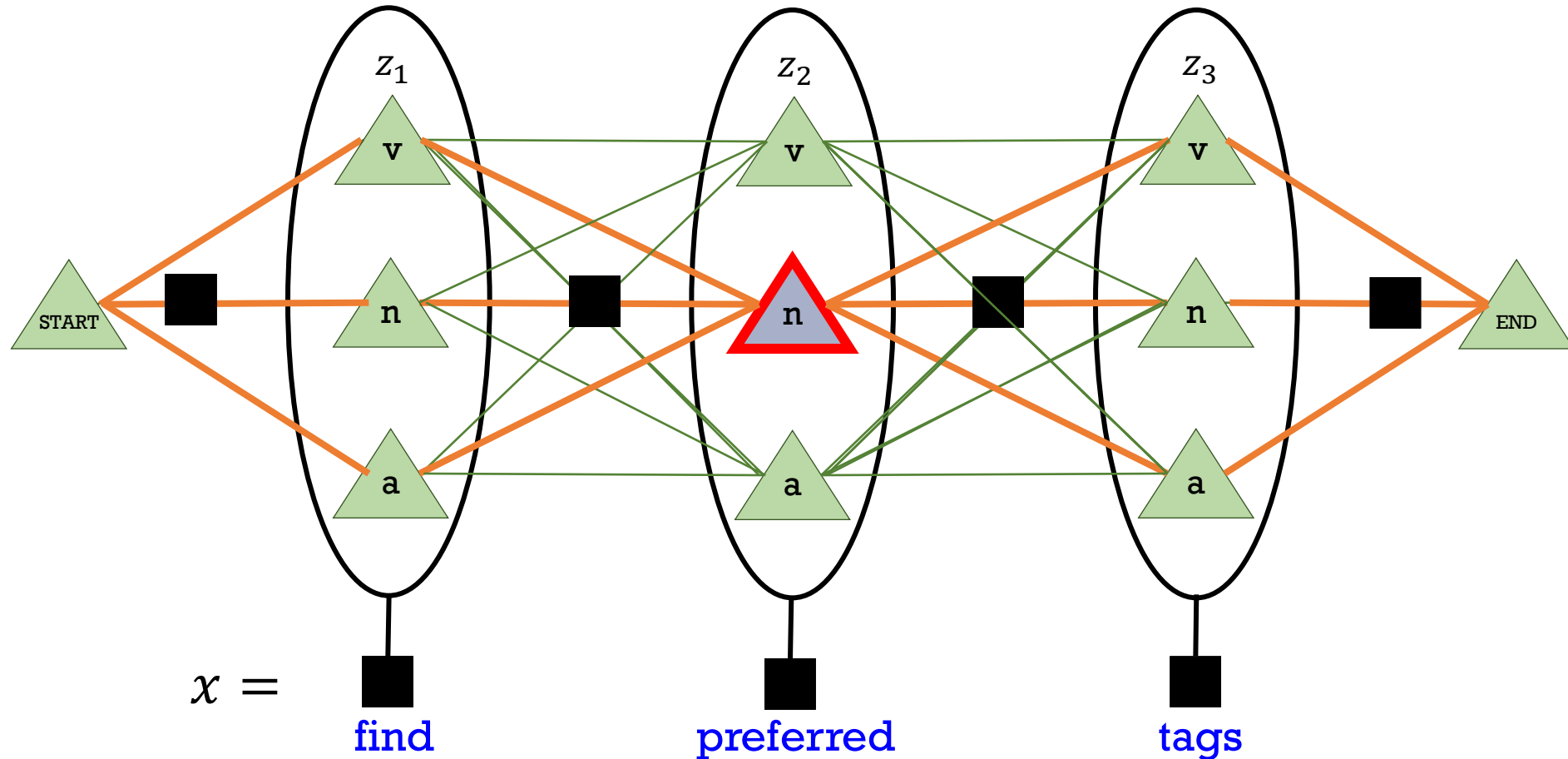
Sum Product (Forward-Backward) Algorithm: Finds Marginals



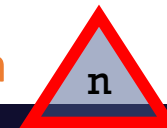
- So $p(\mathbf{v} \mathbf{a} \mathbf{n}) = (1/Z) * \text{product weight of one path}$
- Marginal probability $p(X_2 = v)$
 $= (1/Z) * \text{total weight of all paths through}$



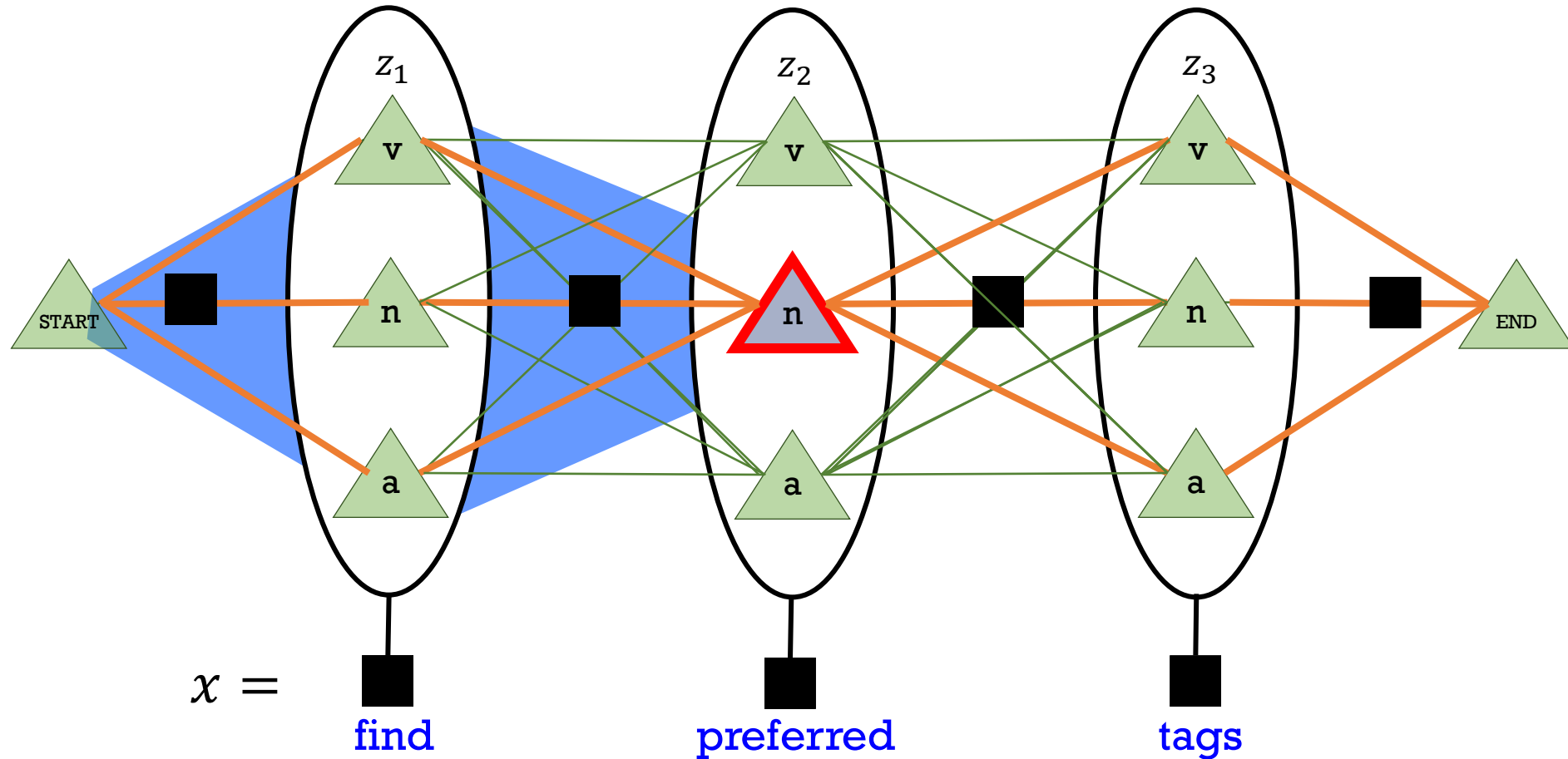
Sum Product (Forward-Backward) Algorithm: Finds Marginals



- So $p(\mathbf{v} \mathbf{a} \mathbf{n}) = (1/Z) * \text{product weight of one path}$
- Marginal probability $p(X_2 = n)$
 $= (1/Z) * \text{total weight of all paths through}$



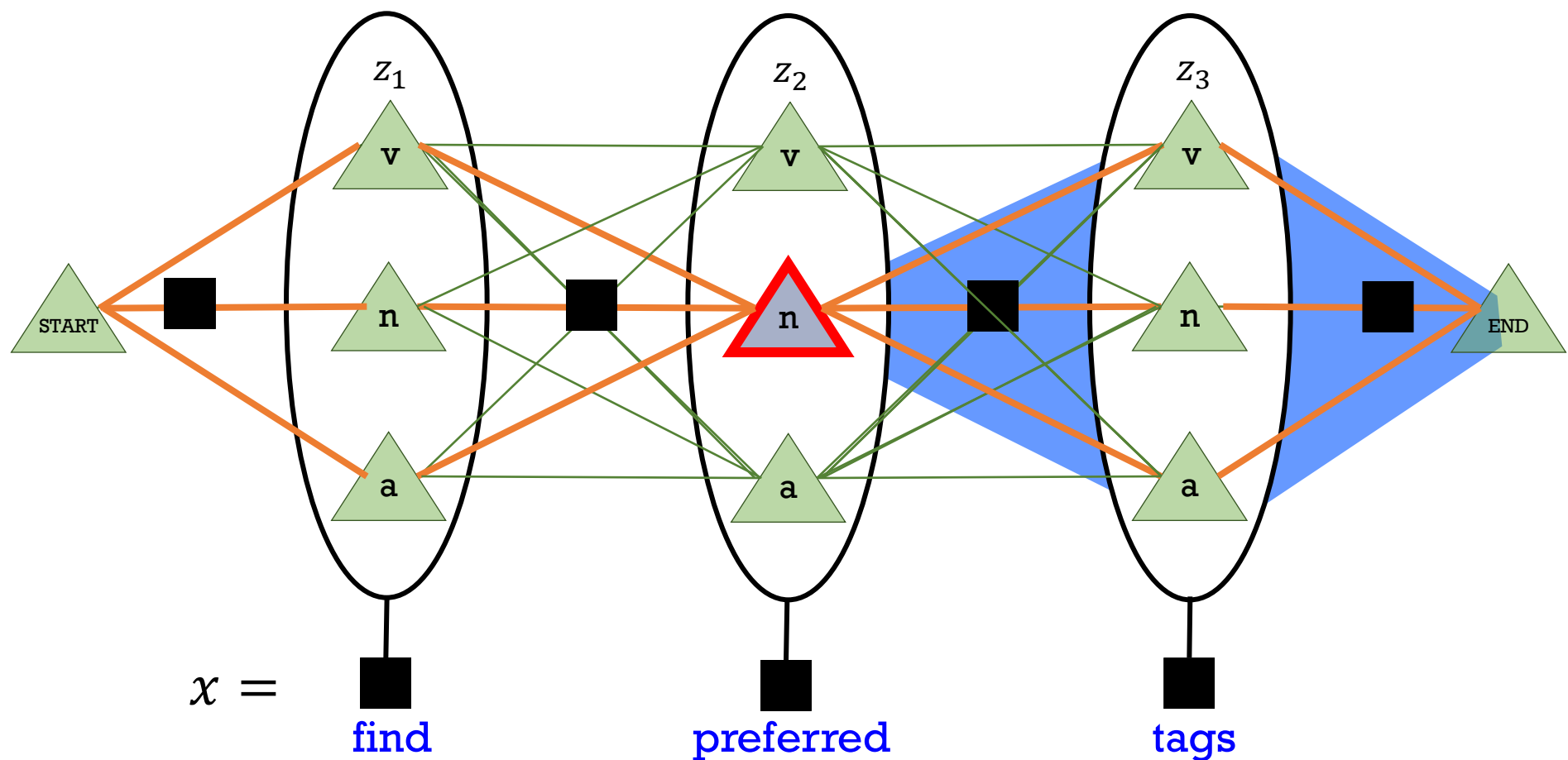
Sum Product (Forward-Backward) Algorithm: Finds Marginals



$\alpha_2(\mathbf{n})$ = total weight of these path *prefixes*

(found by dynamic programming: matrix-vector products)

Sum Product (Forward-Backward) Algorithm: Finds Marginals



$x =$

find

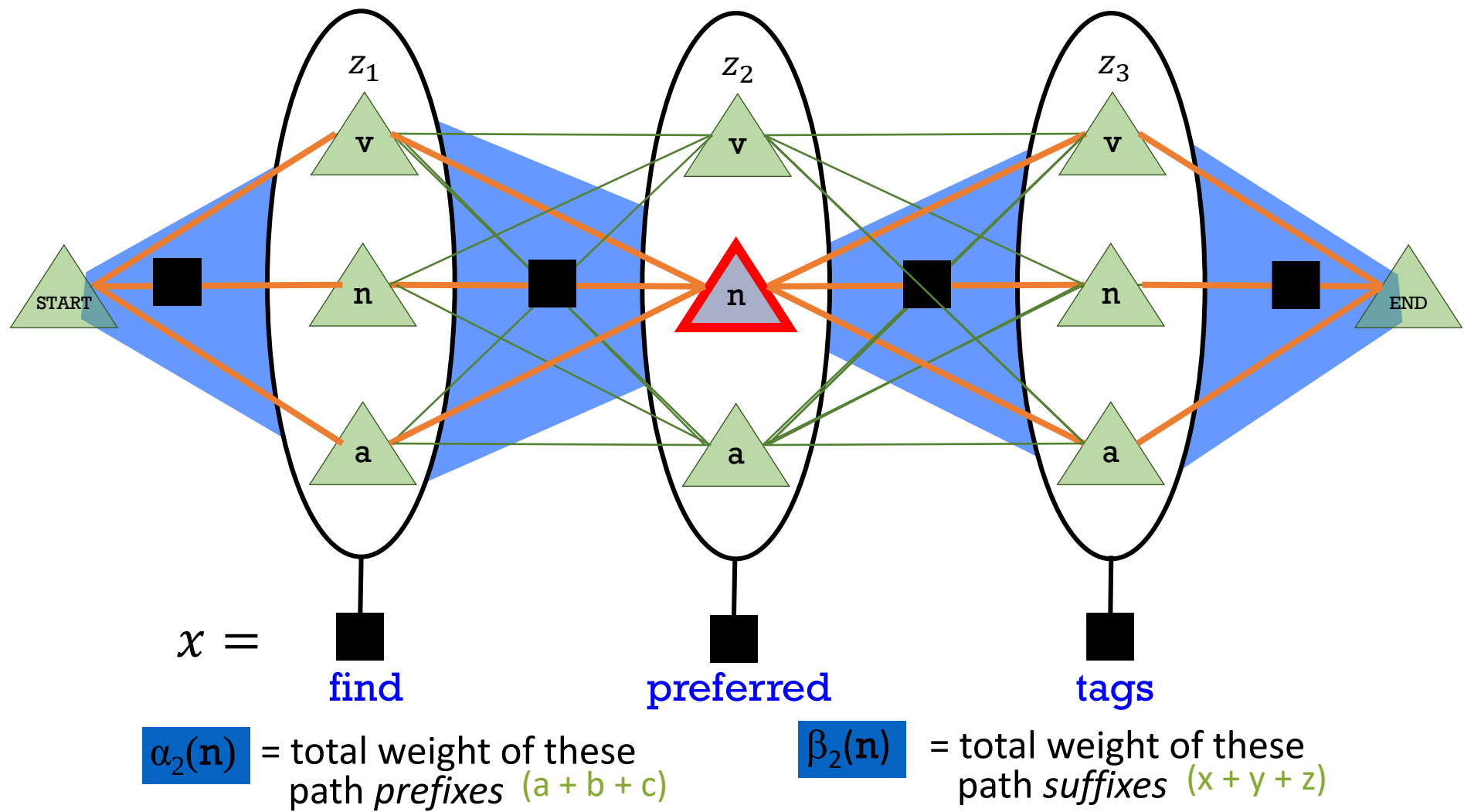
preferred

tags

$\beta_2(n)$ = total weight of these path suffixes

(found by dynamic programming: matrix-vector products)

Sum Product (Forward-Backward) Algorithm: Finds Marginals

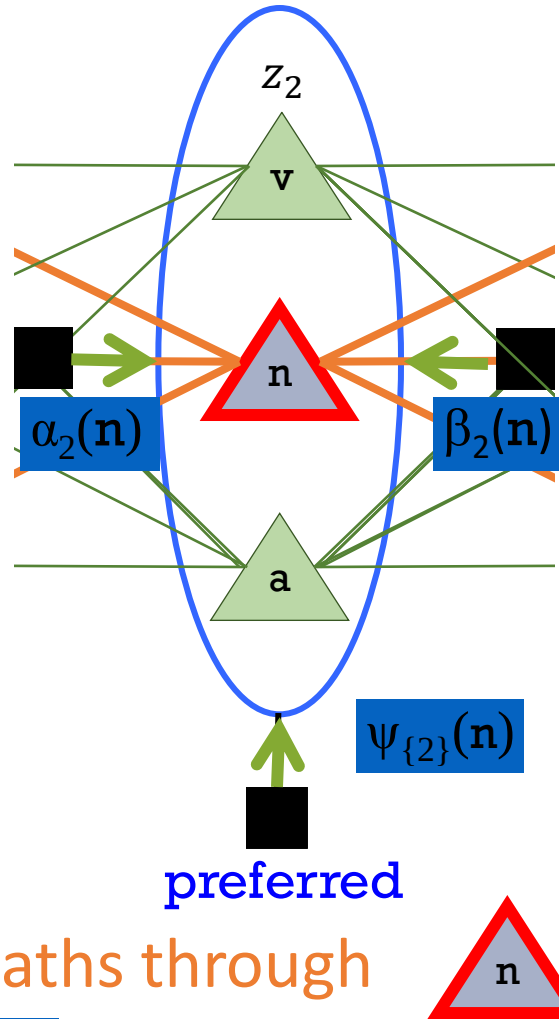


Product gives $ax+ay+az+bx+by+bz+cx+cy+cz$ = total weight of paths

Sum Product (Forward-Backward) Algorithm: Finds Marginals

Oops! The weight of a path through a state also includes a weight at that state.
So $\alpha(\mathbf{n}) \cdot \beta(\mathbf{n})$ isn't enough.

The extra weight is the opinion of the unigram factor at this variable.



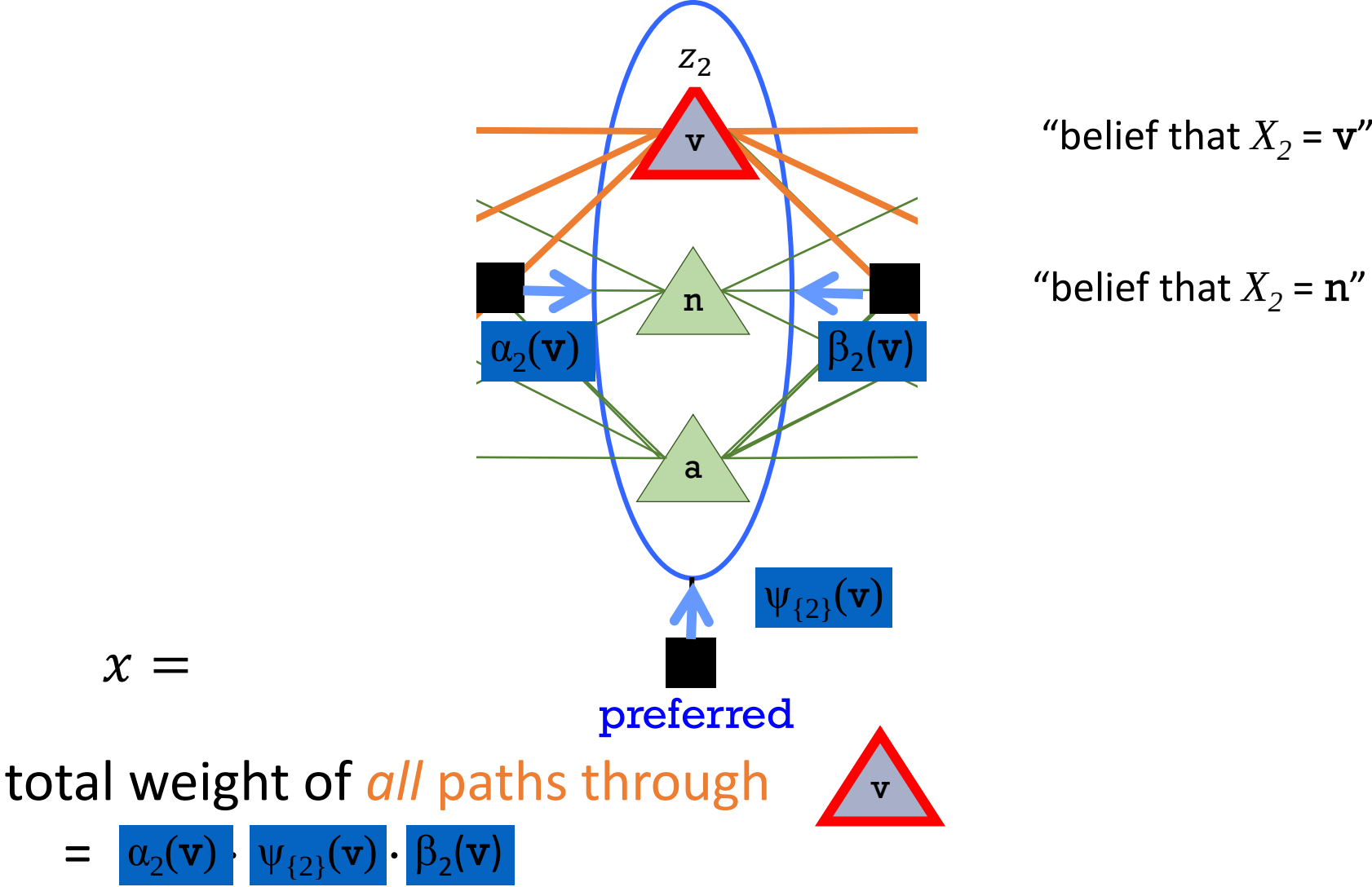
"belief that $X_2 = \mathbf{n}$ "

$\chi =$

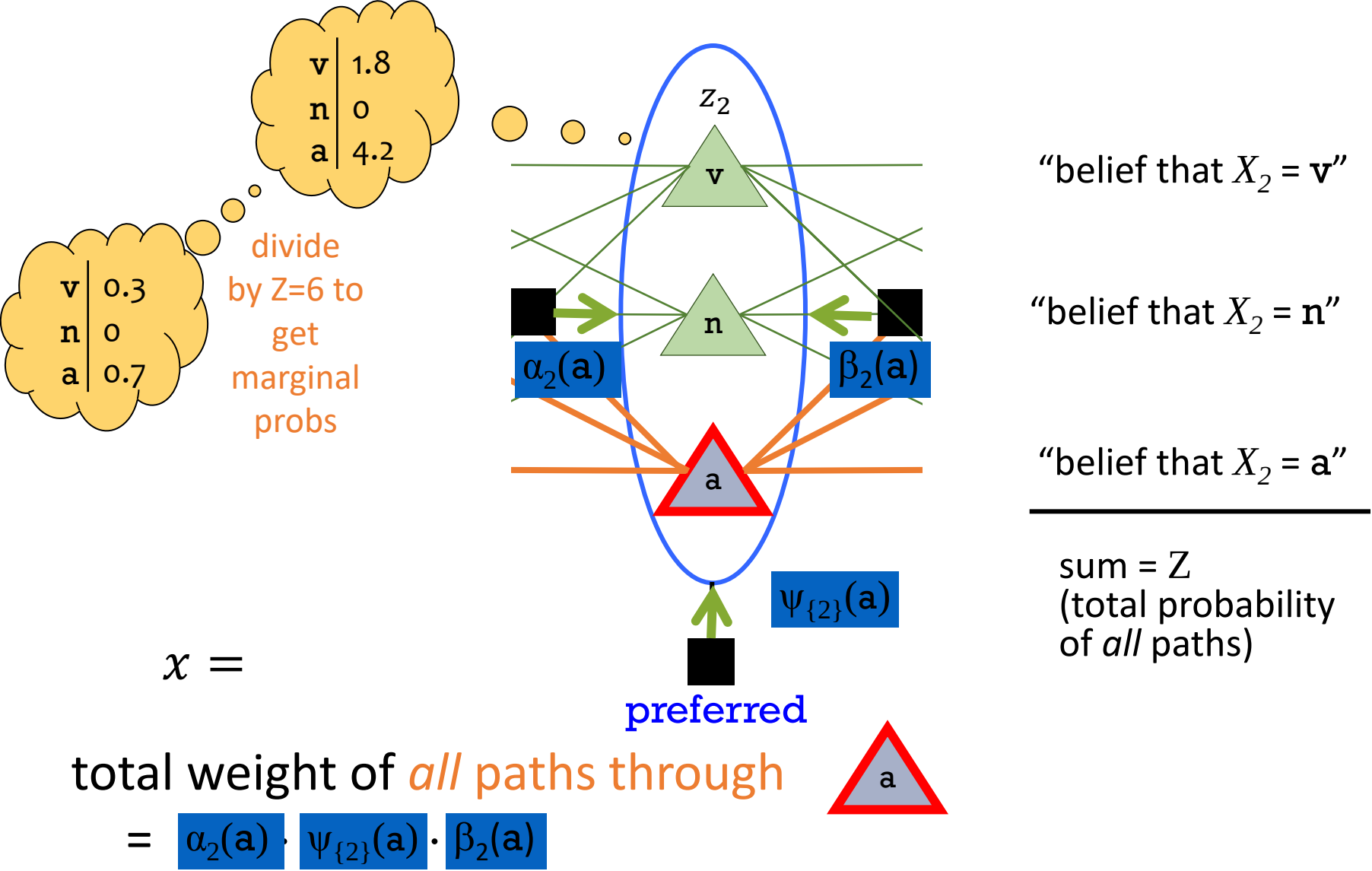
total weight of *all paths through*

$$= \alpha_2(\mathbf{n}) \cdot \psi_{\{2\}}(\mathbf{n}) \cdot \beta_2(\mathbf{n})$$

Sum Product (Forward-Backward) Algorithm: Finds Marginals



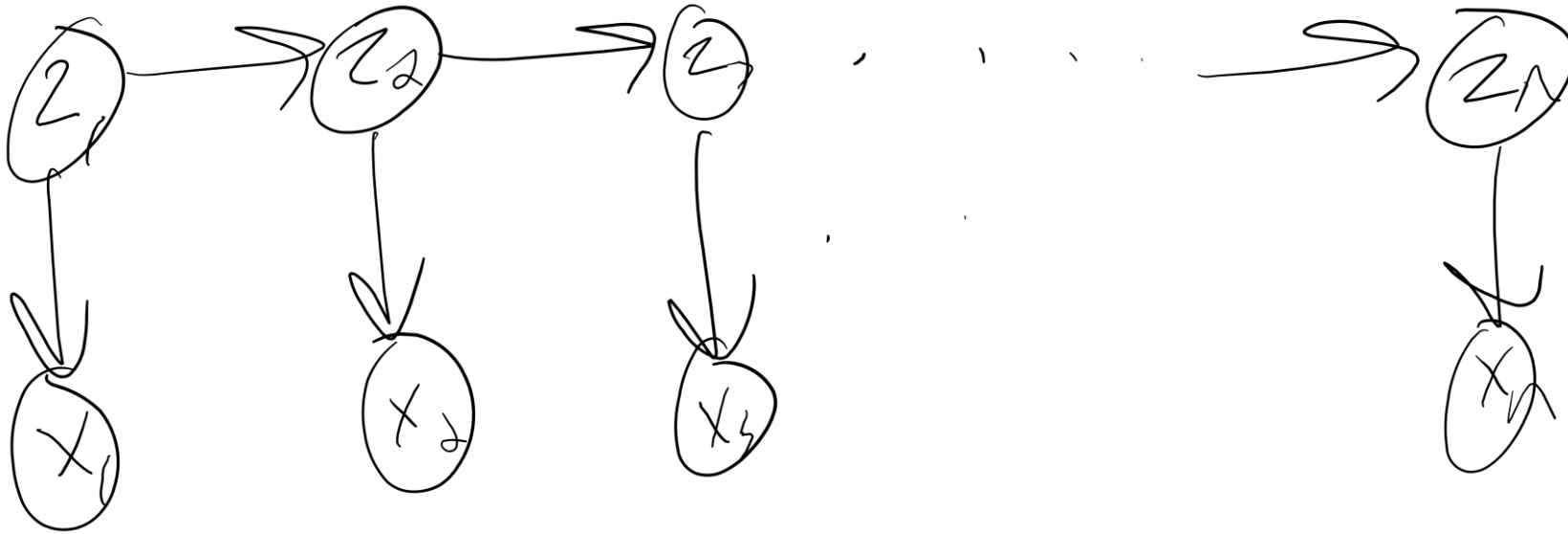
Sum Product (Forward-Backward) Algorithm: Finds Marginals



Dishonest Casino

- A casino has two dice
 - Fair die: $p(1) = p(2) = p(3) = p(4) = p(5) = p(6) = \frac{1}{6}$
 - Loaded die: $p(1) = p(2) = p(3) = p(4) = p(5) = \frac{1}{10}, \quad p(6) = \frac{1}{2}$
- Dealer switches between die with
 - $p(\text{Fair} \rightarrow \text{Loaded}) = 0.01$
 - $p(\text{Loaded} \rightarrow \text{Fair}) = 0.2$
- Given a sequence of rolls from the casino, how can we model this scenario?

Dishonest Casino



- Alpha = forward messages
- Beta = backward messages
- Gamma = marginal probabilities
- [DEMO]

Questions?

